

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Technical Presentation Questions

Course Code:	CSA0303	Course Title:	Data Structures
CO Assessed:	CO1	Assessment:	Assessment Tool 2 – Technical Presentations Questions
Weightage:	10% of CIA	Total Marks:	100
CO1 Statement:	Basics, Data, Data object, Data structure, Abstract Data Types (ADT), realization of ADT in ‘C’, Primitive and non-primitive data structure, Linear and non-linear data structure Arrays & Recursion, Performance analysis and Measurement: Time and space analysis of algorithms, Average, best and worst-case analysis.		
Student Name:	poojitha	Reg. No.:	192511230
Section / Batch:	cse	Date:	

Code of Conduct

I _____aluri poojitha____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- This test contains 3 Concept mapping questions. Total: 100 Marks.
- When a student answer using a concept map, in evaluation the answer should contain following four requirements: Understanding of concepts, Connections between ideas, Logical structure, Clarity of representation.
- No negative marking. Unanswered questions carry zero marks.
- Create a PPT with 15 slides – Laptop permitted for presentation in the class. Take Print out and

submit in the class.

Section / Topic	Focus	Q Nos	Marks
Realization of ADT in 'C'- Array DS- Performance analysis and Measurement: Time and space analysis of algorithms.	Linked List data Structure	1	30
Primitive and non-primitive data structure, Linear and non-linear data structure Arrays	Tries data Structure	2	40
Dynamic linear Data Structures	Queue Data structures	3	40
GRAND TOTAL:		100 Marks	

Annexure A – Technical Presentation

- 1) A music streaming application maintains a playlist where songs can be added, removed, or reordered dynamically. Recommend a suitable data structure and justify based on flexibility and efficiency. Prepare a technical Presentation
- 2) A dictionary application needs to quickly search whether a word exists in its database. Suggest an appropriate data structure and justify based on search performance. Prepare a technical Presentation.
- 3) A task scheduling system assigns jobs based on execution priority levels. Suggest a suitable data structure and justify performance efficiency.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
----------	-----------	---------------------	----------------------	----------------------	---------------------

Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____



DATA STRUCTURES IN PRACTICE

Playlists, dictionary lookups, and priority-based job scheduling



Module 1 — Playlist Management



Module 2 — Dictionary Search



Module 3 — Task Scheduling

AGENDA

Three problems, three structural decisions



01

Dynamic Playlist Management

Add, remove, and reorder songs efficiently



02

Fast Dictionary Word Lookup

Confirm word existence at large scale



03

Priority-Based Task Scheduling

Always execute the most urgent job next



Playlists change shape constantly

Frequent

A user adds a song mid-playlist, or queues one up next, at any

Frequent

Songs are removed from any position — start, middle, or

Constant

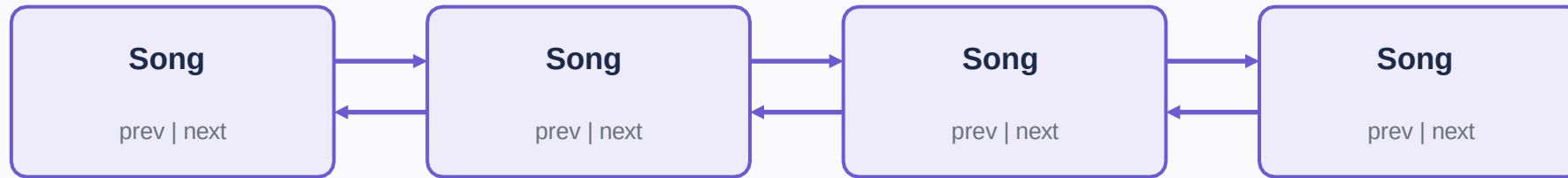
Drag-and-drop reordering and shuffle/repeat change song

Sequential

Playback only ever needs the next / previous song, never random index

Doubly Linked List

(circular variant, for repeat-all mode)



Each node stores pointers to the previous and next song — insertion or deletion is a pointer update, not a data shift

Why it fits

- $O(1)$ insert/delete once the target node is known — no shifting of surrounding songs
- Bidirectional traversal supports both "next" and "previous" track controls directly
- A circular variant naturally supports repeat-all playback without special-case code
- Memory grows and shrinks with the playlist — no pre-allocated capacity to manage

Doubly linked list vs. the alternatives

Operation	Array	Singly Linked List	Doubly Linked List
Insert at head/tail	$O(n)$ shift	$O(1)$	$O(1)$
Insert / delete mid-list	$O(n)$ shift	$O(n)$ to find prev	$O(1)$ once positioned
Move to next song	$O(1)$ index	$O(1)$	$O(1)$
Move to previous song	$O(1)$ index	$O(n)$ rescan	$O(1)$
Reorder (drag-drop)	$O(n)$ shift	$O(n)$	$O(1)$ relink

The doubly linked list is the only structure with $O(1)$ performance on every operation the playlist needs.



Lookup speed is the whole product

Massive

A dictionary may hold hundreds of thousands of

Existence check, not

The only question asked is "does this exact word

High query

Autocomplete and spell-check fire a lookup on nearly every

Read-heavy

Words are looked up far more often than the dictionary is

Hash Table

(with a Trie as a prefix-search extension)



A hash function maps each word to a bucket index; matching words in the same bucket form a short chain

Why it fits

- Average-case $O(1)$ lookup, insert, and delete regardless of dictionary size
- The hash function distributes words evenly, keeping chains short
- No need to keep entries sorted — updates are cheap
- A Trie is worth adding only if prefix queries (autocomplete) become a requirement

Search performance across structures

Operation	Unsorted Array	Sorted Array	Balanced BST	Hash Table
Search (avg)	$O(n)$	$O(\log n)$	$O(\log n)$	$O(1)$
Insert	$O(1)$	$O(n)$ shift	$O(\log n)$	$O(1)$
Delete	$O(n)$	$O(n)$ shift	$O(\log n)$	$O(1)$
Worst-case search	$O(n)$	$O(\log n)$	$O(\log n)$	$O(n)$ rare

At dictionary scale, $O(1)$ average lookup makes the hash table dramatically faster than any comparison-based structure.



MODULE 3 · THE PROBLEM

Execution order depends on priority, not arrival time

Priority, not

A high-priority job submitted last must still run before older, low-priority

Frequent

The scheduler repeatedly asks "which job runs next?" and

Ongoing

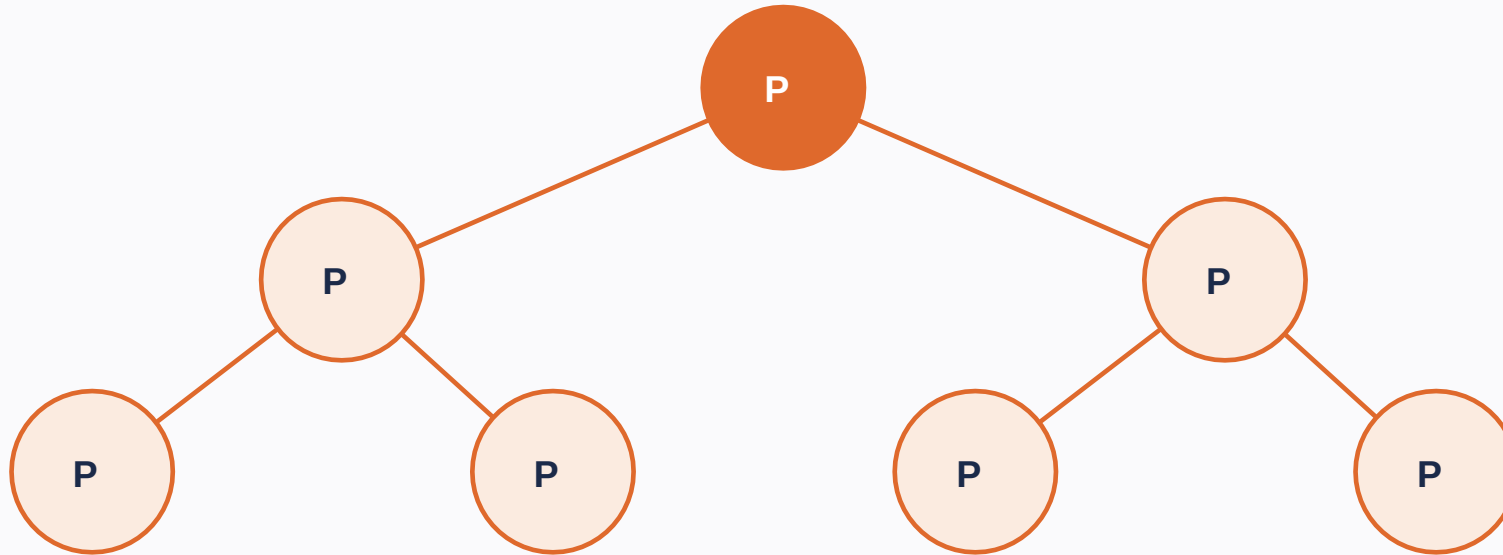
New jobs arrive continuously, each with its own priority

Priorities can

A waiting job may be re-prioritized before it

Priority Queue — Binary Heap

(max-heap: highest priority value sits at the root)



Each parent holds a priority $\geq$ both children — the highest-priority job is always at the root

Why it fits

- Extracting the highest-priority job is $O(1)$ to read, $O(\log n)$ to remove and re-heapify
- Inserting a new job is $O(\log n)$ — just sift the new entry up to its correct level
- Stored compactly in an array — no pointer overhead per node
- Re-prioritizing a waiting job is a single $O(\log n)$ sift-up or sift-down

Binary heap vs. the alternatives

Operation	Unsorted Array	Sorted Array	Binary Heap
Insert new job	$O(1)$	$O(n)$ shift	$O(\log n)$
Find highest priority	$O(n)$ scan	$O(1)$	$O(1)$
Remove highest priority	$O(n)$	$O(1)$ but shifts on insert	$O(\log n)$
Re-prioritize a job	$O(1)$ update, $O(n)$ to find	$O(n)$	$O(\log n)$

The binary heap is the only structure that keeps both insertion and extraction logarithmic — no operation is ever a full scan.

SUMMARY



Playli

Doubly Linked

$O(1)$ insert/delete/reorder,
bidirectional



Scheduler

**Binary Heap
(Priority Queue)**

$O(\log n)$ insert and highest-
priority

Match the structure to the operation that happens most often — not the one that seems most familiar.